SPARKSALES

Team Git & GitHub Workflow Guide

A practical handbook for branch ownership, feature delivery, commits, pushes, pull requests, reviews, merging and the final development-to-main release.

Repository: Campfire-Devs / SparkSales
Branch model: main → develop → feature branches
Branch pattern: feature/<issue-name>
Golden rule: No direct feature work on main.

Purpose

The GitHub backlog supplied for SparkSales contains the team's feature areas and assignees. This guide converts that backlog into one consistent workflow: developers work on their assigned feature branch, push that branch to GitHub, open a PR into develop, resolve review comments, merge after testing, and eventually promote the tested develop branch to main.

Version: September 2026

1. Team Ownership & Branch Map

Issue	Owner	Branch	Expected work
Navigation & Layout	Pulse999	feature/navigation-layout	Responsive sidebar, navbar, dashboard layout, mobile navigation.
Authentication & Business Setup	VJ494	feature/authentication-business-setup	Login, business registration/setup, Terms & Conditions, Privacy Policy.
Dashboard	Pulse999	feature/dashboard	Overview, cards, charts, recent transactions, quick actions.
Sales	Code-Toad29	feature/sales	Record sale, payment method, history, edit/delete, search/filter.
Expenses	Code-Toad29	feature/expenses	Record expense, categories, history, edit/delete, search/filter.
Financial Calculations	Franco232738237	feature/financial-calculations	Revenue, expenses, gross profit/loss, commission, net profit, margin, loss detection.
Reports	Franco232738237	feature/reports	Financial summary, charts/breakdowns, performance, printable/PDF report.
Settings	VJ494	feature/settings	Business profile, team, information, app settings, account deletion.
Finalisation	Whole team	feature/finalisation	Responsive polish, states, validation, toasts, modals, full testing, production build.

Project Setup (#1) is already completed. It is not a feature branch.

2. Branching Rules & What Done Means

Use this structure:

```
main
■■■ develop
■■■ feature/navigation-layout
■■■ feature/authentication-business-setup
■■■ feature/dashboard
■■■ feature/sales
■■■ feature/expenses
■■■ feature/financial-calculations
■■■ feature/reports
■■■ feature/settings
■■■ feature/finalisation
```

Team rules

- main is the stable/release branch. Do not develop features directly on it.
- develop is the integration branch. Completed features enter it through reviewed PRs.
- Work only on the feature branch assigned to you.
- Push your feature branch regularly, but merge it into develop only when the issue is ready.
- Keep commits small and meaningful; avoid one enormous commit at the end.
- Never force-push shared branches without team agreement.
- If multiple people need the same files, communicate first to reduce conflicts.

Definition of done: the issue checklist is complete; the feature works; navigation works; forms validate; relevant loading/empty/error states exist; responsive behaviour is acceptable; the browser console is checked; and the feature has been manually tested.

Current branch naming correction: if the local branch was created as **feature/authentication**, rename it to **feature/authentication-business-setup** so it matches issue #3.

3. Terminal Crash Course

1. Start from an up-to-date develop

```
git status
git switch develop
git pull origin develop
```

2. Create or switch to your assigned branch

```
git switch -c feature/sales
# OR, if it already exists:
git switch feature/sales
```

3. Work, stage and commit

```
git status
git add .
git commit -m "feat: build sales page"
```

4. Push the feature branch

```
git push -u origin feature/sales
```

After the first push, normally use **git push**.

5. Before opening the PR, sync with develop

```
git switch develop
git pull origin develop
git switch feature/sales
git merge develop
git push
```

If a merge conflict appears, resolve the conflicted files carefully, then stage and commit. Do not guess with destructive commands.

I will try and make a CI/CD pipline that shoud check for code conflics and deny a pull reqest to the Developmnet branch if you code doesnt not gel with the rest of the code/file/structure

4. VS Code GUI + Pull Requests

Switch branches

- Click the branch name in the bottom-left status bar.
- Choose develop or your assigned feature branch.
- If the branch exists on GitHub but not locally, choose the remote branch to check it out.

Commit through VS Code

- Open Source Control.
- Review the changed files.
- Enter a clear commit message.
- Stage the intended files.
- Click Commit.

Push through VS Code

- Click Sync Changes / Push.
- Confirm the branch shown is your feature branch.
- Verify the branch appears on GitHub.

Create the Pull Request on GitHub

- GitHub → Pull Requests → New Pull Request.
- Base = **develop**. Compare = your **feature/...** branch.
- Describe what changed, testing performed, and link the issue.

Example PR

```
Title: feat: complete Sales page (#5)

What changed:
- Built Sales page
- Added record-sale form
- Added payment method
- Added history/edit/delete/search

Testing:
- Navigation checked
- Form validation checked
- Responsive layout checked
- Console checked

Closes #5
```

5. Review, Merge & Promote to Main

Reviewer checklist

- Does the PR satisfy the GitHub issue?
- Does it match the SparkSales design?
- Are there console errors or broken states?
- Does the relevant route work?
- Are calculations/data handling correct?
- Is the code understandable and reasonably scoped?
- Has the developer tested it?

After review

- If changes are requested, make them on the same feature branch.
- Commit and push again; the PR updates automatically.
- After approval and testing, merge feature → develop.
- Delete the feature branch after merge if the team agrees. Dont delete branches, incase we have to go back

When do we push to development?

- Push your feature branch whenever useful for backup/collaboration.
- Open the PR when the issue implementation is ready for review.
- Merge into develop only after the team is satisfied that the feature works and review comments are resolved.

Final release flow

```
feature/<name>
↓
PR → develop
↓
Review + integration testing
↓
Develop is stable
↓
PR: develop → main
↓
Final approval
↓
main
```

6. Feature Expectations & Quick Reference

Issue	Minimum outcome before PR
Navigation & Layout	Responsive app shell, sidebar, navbar and mobile navigation.
Authentication & Business Setup	Login, registration/setup and legal-information flows.
Dashboard	Overview cards, charts, transactions and quick actions.
Sales	Record, history, edit/delete, payment method and search/filter.
Expenses	Record, categories, history, edit/delete and search/filter.
Financial Calculations	Revenue, expense, gross profit/loss, commission, net profit, margin and loss detection.
Reports	Financial summaries, charts/breakdowns, performance and printable/PDF output.
Settings	Business profile, team management, business information, app settings and account deletion.
Finalisation	Responsive polish, loading/empty states, validation, toasts, confirmation modals, full testing and successful build.

One-page command reference

Task	Command / action
Check branch	**git branch** or VS Code bottom-left branch selector
Update develop	**git switch develop** → **git pull origin develop**
Create feature	**git switch -c feature/name**
Commit	**git add .** → **git commit -m "feat: ..."**
Push	**git push** or VS Code Sync Changes
PR	GitHub → Pull Requests → New PR → base develop → compare feature branch
Feature merge	Review → test → merge feature → develop
Release	Test develop → PR develop → main → approve → merge

Final rule: the normal direction is feature → develop → main. If main ever receives an urgent fix directly, sync that change back into develop afterwards so the branches do not drift.

Build it. Review it. Integrate it. Release it.